

LAB TEST ANSWER SHEET

MATRICULATION :

FULL NAME :

GROUP:

EXERCISE 1 (5.5 POINTS).

N°	Code 1	Q1. Message	Q2. Correction /Q4. Modification
1	#include <stdio.h>		
2	#include <stdlib.h>		
3	const int con;		
4	int main()		
5	{		
6	bool b = 1;	d	#include <stdbool.h>
7	int i = 0	a	int i = 0 ;
8	char s [10];		
9	s = "bonjour";	f	1 st sol char s [10] = "bonjour"; 2 nd sol #include <stdbool.h> strcpy(s, "bonjour");
10	char ch = 'b';	i	
11	con = 10;	e	const int con = 10;
12	scanf("%d", i);	h	scanf("%d", &i);
13	f = sqrt(i);	c, g	float f = sqrt(i);
14	if (b){		
15	printf("%d %s", i, s, "%f %d", f, con);	j	printf("%d %s %f %d", i, s, , f, con);
16	return 0;		
17	}	b	Mettre } à la ligne 15 printf("%d %s", i, s, "%f %d", f, con);} ou supprimer { à la ligne 14

Q3. Execution Error	a. segmentation fault	b. Buffer overflow
Warning	h	j

EXERCISE 2 (4.5 POINTS).

Q1. Compilation command	1 st sol: gcc code2.c	2 nd sol: gcc -o exe code2.c
Q1. Execution command	a	exe

Code 2	Q2. Trace	
	a = FCT(tab, i, n);	PROC(tab, i, a); //print Tab
<pre> int FCT(int tab [], int d, int f) { int a = d; for (int i = d+1; i < f; i++) if (tab[i] < tab[a]) { a = i; } return a; } void PROC(int tab [], int i, int j) { int tmp = tab [i]; tab [i] = tab [j]; tab [j] = tmp; } int main() { int tab[] = {5, 3, 2, 1, 4, 7, 2}; int a, n = sizeof(tab) / sizeof(tab[0]); for (int i = 0; i < n-1; i++) { a = FCT(tab, i, n); PROC(tab, i, a); } return 0; } </pre>	a = 3	Tab = {1, 3, 2, 5, 4, 7, 2} (i = 0)
	a = 2	Tab = {1, 2, 3, 5, 4, 7, 2} (i = 1)
	a = 6	Tab = {1, 2, 2, 5, 4, 7, 3} (i = 2)
	a = 6	Tab = {1, 2, 2, 3, 4, 7, 5} (i = 3)
	a = 4	Tab = {1, 2, 2, 3, 4, 7, 5} (i = 4)
	a = 6	Tab = {1, 2, 2, 3, 4, 5, 7} (i = 5)

Q3. Code role:

sort array using selection sort (*FCT* : find a minimum, *PROC* : swap values of cell *i* and *a*)

EXERCISE 3 (5 POINTS).

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

int mirror (int n) {
    int m = 0;
    while (n > 0) {
        m = m * 10 + (n % 10);
        n = n / 10;
    }
    return m;
}

bool palindrome (int N) { return(N == mirror(N)); }

void FirstPalindrome(int N, int * FP, int * step)
{
    *step = 0; *FP = N;
    while(!palindrome(*FP)) {
        *FP = *FP + mirror(*FP);
        *step = *step + 1;
    }
}

int main() {
    int N, FP, step;
    printf("Enter a positive starting number N = ");
    scanf("%d", &N);
    FirstPalindrome(N, &FP, &step);
    printf("\n For this number, we reach the palindrome %d in %d steps.", FP, step);
    return 0;
}
```